Project 357 — Review Guide

Kubernetes-Based Auto-Scaling Framework for Cloud Applications
PRAH — Priority-based Rule Arbitration with Hysteresis

> **Alliance University, Bengaluru** | Department of Computer Science & Engineering
> Design Project-I | Project ID: 357 | Milestone 1 (First 50% Functional Core)
> Deterministic, rule-based scaling. No machine learning, no forecasting, no predictive models.

1. Elevator Pitch

> **One line:** A deterministic, policy-driven autoscaler for Kubernetes that replaces HPA's single-metric ratio math with multi-rule priority arbitration, hysteresis, cooldowns, hard guardrails, and a complete audit trail.

Problem. Native Kubernetes HPA relies on simple ratio calculations. It cannot arbitrate competing metrics, cannot explain why it scaled, and can oscillate between scale-out and scale-in under noisy telemetry.

Solution. PRAH inserts an explainable decision layer above the Kubernetes API: validate telemetry, evaluate declarative YAML policies, arbitrate by priority, suppress flapping, clamp against operational limits, then patch the Deployment scale subresource and verify the resulting state.

2. Architecture — One Control Cycle

```
Prometheus ──> Telemetry ──> Policy Engine ──> PRAH Decision ──> Guardrails ──> Strategy Coordinator
──> K8s Executor ──> Audit Logger ──> Dashboard
            validate       thresholds +      priority, tie-    min/max,      HPA conflict check
patch scale        JSONL          REST API
            freshness      duration window   break, hysteresis step clamp,                          +
verify state
                                              cooldown          dry-run
```

Stage	File	Responsibility
Orchestrator	`controller/main.py`	Runs the loop and wires all modules together
Telemetry	`controller/telemetry.py`	PromQL query, NaN/Inf rejection, staleness, MISSING vs ERROR
Policy	`controller/policy_engine.py`	YAML parse/validate, operators, duration persistence
PRAH	`controller/decision_controller.py`	Priority sort, tie-break, hysteresis, cooldown
Guardrails	`controller/guardrails.py`	min/max replicas, step clamp, dry-run vs live
Coordinator	`controller/strategy_coordinator.py`	Rejects execution if a native HPA owns the deployment
Executor	`controller/kubernetes_executor.py`	Patch scale subresource and verify new state
Logger	`controller/decision_logger.py`	Thread-safe JSONL audit plus in-memory ring buffer
Dashboard	`dashboard/app.py`	Flask REST API and 7-page operator console
Workload	`app/app.py`	Sample Flask app with `/metrics`, `/load`, traffic generator
Policies	`policies/scaling-policies.yaml`	6 rules, guardrails, fallback configuration

3. The PRAH Algorithm

The name stands for **P**riority-based **R**ule **A**rbitation with **H**ysteresis. The four deterministic steps live in `compute_decision()`:

1. **Telemetry safety gate** — if any metric is unusable, return `HOLD` immediately (priority 999).
2. **Priority arbitration** — sort matched proposals by priority descending; highest priority wins.
3. **Tie-break** — equal priority with opposite directions: `SCALE_OUT` wins to protect availability.
4. **Hysteresis and cooldown** — block a direction flip within 30 s; block repeat actions during the policy cooldown (60 s default).

controller/decision_controller.py:192 — priority arbitration and tie-breaking

```
sorted_proposals = sorted(matched_proposals, key=lambda p: p.priority, reverse=True)
top_priority = sorted_proposals[0].priority
top_candidates = [p for p in sorted_proposals if p.priority == top_priority]

if scale_out_candidates and scale_in_candidates:
    # Tie-breaking rule: Scale-Out wins to preserve availability & SLAs
    scale_out_candidates.sort(key=lambda p: p.replica_delta, reverse=True)
    chosen_proposal = scale_out_candidates[0]
```

controller/decision_controller.py:242 — anti-flapping protection

```
if is_direction_flip and time_since_last_action < self.min_direction_flip_delay_seconds:
    return Decision(
        action="HOLD",
        explanation="PRAH Hysteresis Protection: Opposing action requested too soon. ...",
        ...
    )
```

4. Key Code to Show

(a) Telemetry never fabricates values — `telemetry.py:65`

The strongest engineering-rigor point: missing or malformed data is never coerced to zero.

```
def is_usable(self) -> bool:
    return (
        self.status in (MetricStatus.VALID, MetricStatus.SIMULATED)
        and self.value is not None
        and isinstance(self.value, (int, float))
        and math.isfinite(self.value)   # rejects NaN / +Inf / -Inf
    )
```

Statuses: `VALID, MISSING, STALE, ERROR, SIMULATED, INVALID, UNAVAILABLE` (`telemetry.py:35`). Staleness threshold is 60 s (`telemetry.py:315`).

(b) Policies are declarative YAML — `policies/scaling-policies.yaml:49`

```
- id: "CPU_HIGH_LOAD"
  priority: 80
  metric: "cpu_utilization_percent"
  operator: ">"
  threshold: 75.0
  duration_seconds: 5          # must stay breached for 5 seconds
  action: "SCALE_OUT"
  replica_delta: 1
  cooldown_seconds: 60
```

controller/policy_engine.py:264 — duration persistence per rule

```
if rule.id not in self._breach_timestamps:
    self._breach_timestamps[rule.id] = current_time
elapsed = current_time - self._breach_timestamps[rule.id]
duration_met = elapsed >= rule.duration_seconds
```

(c) Guardrails are separate from decisions — `guardrails.py:115`

The decision may say "scale +2"; guardrails may clamp it to +1 or reject it at a boundary.

```
if proposed_action == "SCALE_OUT" and final_delta > self.max_scale_step_up:
    final_delta = self.max_scale_step_up
    clamped = True

if authorized_target > self.max_replicas:
    if current_replicas >= self.max_replicas:
        return GuardrailResult(approved=False, action="REJECTED", ...)
```

(d) Executor verifies every mutation — `kubernetes_executor.py:317`

Success is never reported just because the request was accepted.

```
self._apps_v1_api.patch_namespaced_deployment_scale(
    name=self.deployment_name,
    namespace=self.namespace,
    body={"spec": {"replicas": target_replicas}},
)

# Poll until spec.replicas matches, otherwise report VerificationTimeout
while time.time() < deadline:
    if self.get_workload_state().desired_replicas == target_replicas:
        confirmed_replicas = target_replicas
        break
    time.sleep(1.0)
```

(e) One full cycle — `main.py:139`

```
workload_state = self.executor.get_workload_state()          # 1. read state
metrics = self.fetch_current_telemetry()                     # 2. collect
decision = self.decision_controller.compute_decision(...)    # 3. decide (PRAH)
guardrail_res, exec_res = self.strategy_coordinator.coordinate_and_execute(...)  # 4-5
if exec_res.success and guardrail_res.action == "EXECUTE":
    self.decision_controller.record_action_executed(...)     # 6. start cooldown
self.decision_logger.log_cycle(...)                          # 7. audit
```

(f) Dashboard API — `dashboard/app.py:326`

```
@app.route("/api/evaluate", methods=["POST"])
def trigger_evaluation():
    record = controller.run_single_cycle()
    return jsonify({"success": True, "cycle_result": record})
```

Additional endpoints: `/api/status`, `/api/metrics`, `/api/decisions`, `/api/history`, `/api/policies`, and `/api/controls/{live-execution, simulate-metric, clear-simulation, reset-cooldown}`.

Seven console pages (`dashboard/app.py:52-86`): `/overview`, `/workloads`, `/telemetry`, `/policies`, `/decisions`, `/history`, `/system-health`.

5. Demo Script for Review Day

1. Start Prometheus on 9090, sample app on 8080, dashboard on 5055.
2. Open `/overview`; show `HOLD` at nominal load (CPU 45%).
3. Set CPU to 85% via `/api/controls/simulate-metric`, then call `/api/evaluate`: **SCALE_OUT**, target +1, cooldown active.
4. Evaluate again immediately: **HOLD (cooldown)** — proof of anti-flapping.
5. Reset cooldown, run the tie-break scenario (CPU 88% with memory 20%): **SCALE_OUT wins**.
6. Set CPU to 15% at 1 replica: **REJECTED (min_replicas)**.
7. Mark a metric `MISSING`: **HOLD (telemetry fallback)** — the system never scales on bad data.
8. Open `/history` and show the JSONL record containing metrics, decision, guardrail, and execution.
9. Note that live execution requires both Kubernetes and Prometheus to be reachable (`dashboard/app.py:340`) and is gated by a confirmation modal.

6. Tests and Validation

- Seven test files under `tests/`: telemetry, policy engine, decision controller, hysteresis, guardrails, Kubernetes executor, dashboard API.

- 53 test functions are defined; the project documentation reports 59 passing because of parametrized cases. Run with `python -m pytest tests/ -v`.
- Graceful degradation is explicitly tested: with the cluster offline, the executor reports `connected=False` and falls back to 2 simulated replicas (`kubernetes_executor.py:143`).

7. Likely Viva Questions

Question	Answer
Why not just use HPA?	HPA is ratio-based, effectively single-metric, and opaque. PRAH arbitrates multiple metrics, applies priorities, and logs a human-readable reason for every action.
Is any of this machine learning?	No. Every branch is a comparison, sort, clamp, or timer, so results are fully reproducible.
Where is the hysteresis band?	It is implemented as a direction-flip delay (`min_direction_flip_delay_seconds = 30`). `hysteresis_band_percent` is loaded by config but not yet applied as a value dead-band — describe this as a Milestone-2 refinement rather than overclaiming.
What happens with bad telemetry?	The `is_usable()` gate forces `HOLD` at priority 999. No fabricated zeros.
What if two rules conflict?	Priority descending, then SCALE_OUT over SCALE_IN, then largest or most conservative delta depending on direction.
What prevents runaway scaling?	`max_replicas`, `max_scale_step_up/down`, cooldown timers, and the single-owner native-HPA check.
What is deferred?	Empirical PRAH-vs-HPA benchmarking, fault injection, KEDA/custom metrics integration, and VPA coordination (README section 10).

8. Sixty-Second Opening Statement

> Project 357 is a deterministic autoscaling control loop for Kubernetes. It pulls telemetry from Prometheus, validates freshness and numeric sanity, and evaluates YAML-defined policies with priority and persistence windows. PRAH then arbitrates conflicts, applies anti-flapping and cooldown rules, passes the result through hard guardrails, and finally patches the Deployment scale subresource with post-verification. Every cycle is written to a JSONL audit log and surfaced in a seven-page operator dashboard. There is no machine learning: each decision is reproducible and explainable.

Note on docs: `README.md`, `MODULE_CONTRACTS.md`, and `PROJECT_BASELINE.md` were being edited concurrently while this guide was generated. Code references were read from the current source; line numbers may shift slightly if those files change again.